

Task 04 — API Versioning

The task asks you to implement and compare:

`/api/v1/users`

`/api/v2/users`

and document **backward compatibility**.

Since you are working with Spring Boot, do it like this.

Step 1 — Create UserControllerV1.java

Create this file in your controller package:

```
package com.example.userservice.controller;

import org.springframework.web.bind.annotation.*;

import java.util.Arrays;
import java.util.List;

@RestController
@RequestMapping("/api/v1/users")
public class UserControllerV1 {

    @GetMapping
    public List<String> getUsers() {
        return Arrays.asList(
            "Raju",
            "Kiran",
            "Suresh"
        );
    }

    @GetMapping("/{id}")
    public String getUser(@PathVariable Long id) {
```

```
        return "User V1 - ID: " + id;
    }
}
```

Step 2 — Create UserControllerV2.java

```
package com.example.userservice.controller;

import org.springframework.web.bind.annotation.*;

import java.util.Arrays;
import java.util.List;

@RestController
@RequestMapping("/api/v2/users")
public class UserControllerV2 {

    @GetMapping
    public List<UserResponse> getUsers() {
        return Arrays.asList(
            new UserResponse(1L, "Raju", "raju@gmail.com"),
            new UserResponse(2L, "Kiran", "kiran@gmail.com"),
            new UserResponse(3L, "Suresh", "suresh@gmail.com")
        );
    }

    @GetMapping("/{id}")
    public UserResponse getUser(@PathVariable Long id) {
        return new UserResponse(
            id,
            "Raju",
            "raju@gmail.com"
        );
    }
}
```

Step 3 — Create UserResponse.java

```
package com.example.userservice.controller;

public class UserResponse {

    private Long id;
    private String name;
    private String email;

    public UserResponse(Long id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }
}
```

Step 4 — Test V1

Run your Spring Boot application.

Open:

<http://localhost:8080/api/v1/users>

You should get:

```
[  
  "Raju",  
  "Kiran",  
  "Suresh"  
]
```

For a particular user:

GET <http://localhost:8080/api/v1/users/1>

Response:

User V1 - ID: 1

Step 5 — Test V2

Open:

<http://localhost:8080/api/v2/users>

Response:

```
[  
  {  
    "id": 1,  
    "name": "Raju",  
    "email": "raju@gmail.com"  
  },  
  {  
    "id": 2,  
    "name": "Kiran",  
    "email": "kiran@gmail.com"  
  },  
  {
```

```
"id": 3,  
"name": "Suresh",  
"email": "suresh@gmail.com"  
}  
]
```

For a particular user:

GET <http://localhost:8080/api/v2/users/1>

Response:

```
{  
  "id": 1,  
  "name": "Raju",  
  "email": "raju@gmail.com"  
}
```

Step 6 — Compare V1 and V2

Feature	V1	V2
Endpoint	/api/v1/users	/api/v2/users
Response	Simple user information	Detailed user information
ID	Not included in list response	Included
Email	Not included	Included
Existing clients	Supported	New clients can use this
Compatibility	Old API remains available	New version

Backward compatibility

The important point is:

Do not remove /api/v1/users when introducing /api/v2/users.

Existing applications using:

`/api/v1/users`

will continue working.

New applications can use:

`/api/v2/users`

This demonstrates **backward compatibility**.

Task 4 result:

You should have:

User Service

|

└─ controller

├─ UserControllerV1.java

├─ UserControllerV2.java

└─ UserResponse.java

Then test both endpoints in Postman or your browser and take screenshots for your task submission.

